PKCERT AI & Software Development Internship

Task 23 – Convolutional Neural Networks (CNNs): Convolution, Pooling & Architecture Design

Objective:

Develop a rigorous, from-first-principles understanding of convolutional neural networks. Interns will implement 2D convolution and pooling operations manually, derive and implement forward and backward propagation through convolutional and pooling layers by hand (without autograd for the core computation), design and justify a small CNN architecture, and train it on a real image dataset — analyzing the effect of architectural choices on performance and receptive field.

Total Marks: 100

Part A – Convolution Fundamentals (20 Marks)

- Implement 2D convolution from scratch using only NumPy, supporting multi-channel inputs, configurable stride, padding, and dilation (no scikit-image, scipy.signal, TensorFlow, or PyTorch for this part).
- Derive the output shape formula for a convolution given input size, kernel size, stride, padding, and dilation, and verify your implementation against it.
- Implement and compare 'valid', 'same', and 'full' padding modes; show the output dimensions produced by each on a sample input.
- Apply at least three fixed kernels (e.g., edge detection, Gaussian blur, sharpen) to a sample grayscale image, visualize the results, and explain each kernel's effect mathematically in terms of the convolution operation.
- Implement an im2col-based (matrix-multiplication) version of convolution and benchmark it against your naive nested-loop implementation for correctness (identical output) and speed.

Part B – Pooling & Regularization (15 Marks)

- Implement max pooling and average pooling from scratch using only NumPy, including both the forward pass and the backward pass (gradient computation).
- Derive and implement the backward pass for each: gradient routing to the max-argument for max pooling, and even gradient distribution for average pooling.
- Compare pooling versus strided convolution as downsampling mechanisms; discuss the trade-offs in terms of information loss, translation invariance, and parameter count.
- Implement Batch Normalization and Dropout from scratch (forward and backward passes) and explain their role in stabilizing and regularizing CNN training.

Part C – Architecture Design & Backpropagation Mini-Project (45 Marks)

- Select a real image classification dataset (e.g., a subset of CIFAR-10 or Fashion-MNIST), different from datasets used in previous tasks, suitable for a small CNN.
- Manually derive the forward propagation equations (matrix/tensor form) for a CNN with at least two convolutional layers, pooling layers, and a fully connected classification head, and include the derivation in your submission.

- Manually derive the backpropagation gradients (via the chain rule) through the convolutional and pooling layers, including gradients with respect to kernels, biases, and layer inputs, and include the full derivation.
- Implement forward and backward propagation from scratch using only NumPy for the convolutional and pooling layers — no autograd frameworks (TensorFlow/PyTorch/Keras) permitted for this step.
- Train your network using mini-batch gradient descent with momentum, tracking and plotting the loss and accuracy curves over epochs.
- Evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix, and compare performance against a PyTorch or TensorFlow CNN baseline of similar architecture trained on the same data.
- Run an ablation study comparing at least two architecture variants (e.g., kernel size, network depth, presence or absence of pooling layers) and analyze the effect on performance and effective receptive field.
- Visualize the learned filters of the first convolutional layer and the resulting feature maps for a sample input.

Part D – Analysis & Documentation (20 Marks)

- Save your final trained model's parameters (weights and biases) using pickle or joblib.
- Summarize the full pipeline: architecture chosen, layer configuration, activation functions used, and the reasoning behind each design choice.
- Compute and report the effective receptive field size of your final architecture, showing the calculation.
- Summarize key findings from your Part C ablation study.
- Discuss at least two challenges faced while deriving or implementing convolution/pooling backpropagation, and how you resolved them.
- Reflect on the limitations of a manually implemented CNN compared to a production-grade deep learning framework (e.g., compute efficiency, cuDNN-level optimizations, automatic differentiation).

Assessment Breakdown

Assessment Area	Marks
Convolution Fundamentals	20
Pooling & Regularization	15
Architecture Design & Backpropagation (Mini-Project)	45
Analysis & Documentation	20
Total	**100**

Note: For Parts A, B, and C's core computations, use of high-level automatic-differentiation frameworks (TensorFlow, PyTorch, Keras) is not permitted — the intent is to demonstrate a working understanding of the underlying mathematics of convolution, pooling, and backpropagation. Frameworks may be used only for the comparative baseline in Part C.